MÓDULO 5 · FRONTEND & TESTING

# Estrategia de Testing y Arquitectura de Software

Implementación práctica en React, Jest y análisis comparativo de arquitecturas

# Estructura de la Entrega

### Entorno y Suite

4 niveles de test automáticos.

### Monolito vs Microservicios

Impacto en integración.

### MVC vs Hexagonal

Aislamiento unitario sin acoplamiento.

# Suite de Pruebas Implementada

01. UNITARIA

### TodoItem

Renderizado aislado de tarea.

02. SNAPSHOT

### TodoList

Congelación estructural DOM.

03. INTEGRACIÓN

### TodoApp

Input y agregación reactiva.

04. ACEPTACIÓN

### Flujo Completo

Ciclo completo de completado.

# Resultado de Ejecución

**100%**

**4 suites de pruebas superadas en Jest y JSDOM**

Garantía de regresión cero en renderizado e interacción de usuario

# Monolito vs Microservicios en Testing

## Arquitectura Monolítica

**Ventaja:** Integración directa en memoria.

**Datos:** Transacciones en base única.

**Reto:** Suites pesadas al escalar.

Pruebas de integración triviales y rápidas.

## Microservicios

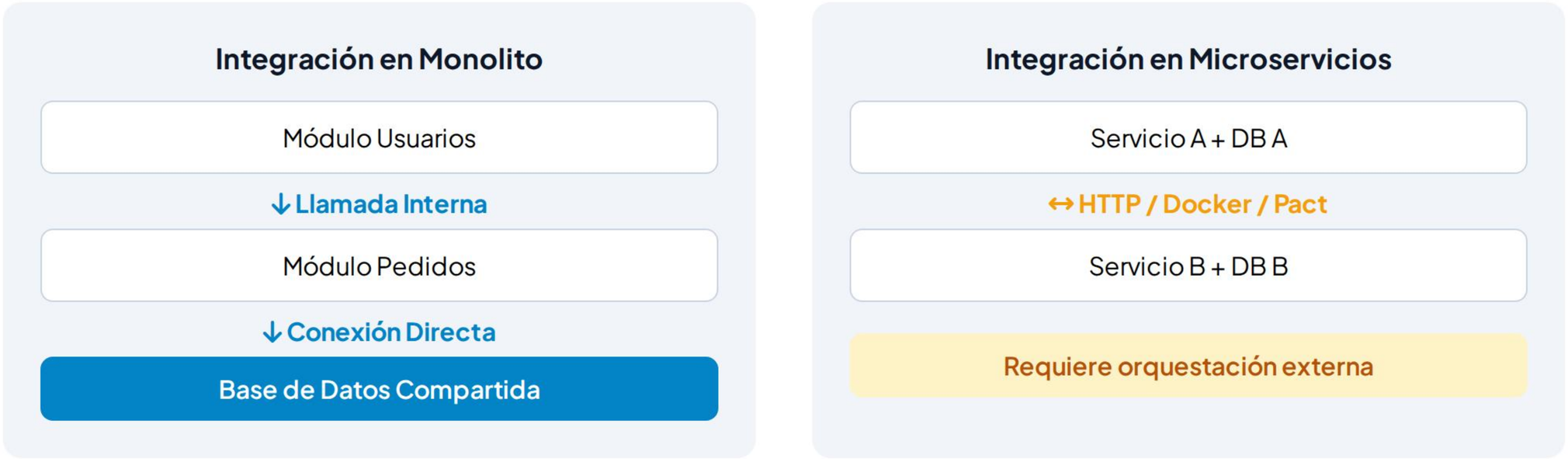
**Ventaja:** Tests unitarios desacoplados.

**Infraestructura:** Requiere Docker y red.

**Contratos:** Necesidad de Contract Testing (Pact).

Alta complejidad en pruebas integradas.

# Flujo de Integración: Monolito vs Red



# MVC vs Arquitectura Hexagonal

## Patrón MVC

**Acoplamiento:** Controladores ligados a la web.

**Dependencia:** Modelos unidos al ORM.

**Test:** Difícil aislar lógica de negocio pura.

Requiere mocks pesados de frameworks.

## Arquitectura Hexagonal

**Dominio:** Lógica pura sin frameworks.

**Puertos:** Interfaces claras de comunicación.

**Test:** Inyección de mocks ultrarrápida.

Testabilidad unitaria óptima y aislada.

# Aislamiento en Arquitectura Hexagonal




Los tests unitarios interactúan directamente con los puertos sin necesidad de infraestructura externa.

# Conclusión de Diseño

> **La arquitectura del software determina directamente la facilidad, el costo y la velocidad de su estrategia de pruebas.**

El desacoplamiento temprano ahorra tiempo de testing durante todo el ciclo de vida del producto.



# Código Fuente en GitHub

Repositorio con la implementación completa de la Todo List y sus 4 niveles de pruebas comentadas

```
https://github.com/tu-usuario/proyecto-testing
```